Revisão

Nivelamento

Instrutor: Ralf Lima

Lógica e Algoritmos

Mensagens

```
// Exibir mensagem
print("Hello World!")
```

Variáveis

```
String -> Textual
Char   -> Letra/Caractere

Int    -> Números inteiros
Float  -> Suporte entre 6 e 7 casas decimais
Double -> Double entre 15 e 16 casas decimais
UInt   -> Não aceita números negativos

Bool   -> Boolean/Lógica (true / false)

Any       -> Flexível
AnyObject -> Classes
```

Concatenar

```
// Concatenação
var nome = "Ralf"
print("Bom dia " + nome) // Maneira incorreta
print("Bom dia \(nome)") // Maneira correta
```

Operadores aritméticos

```
+  -> SOMA
-  -> SUBTRAÇÃO
*  -> MULTIPLICAÇÃO
/  -> DIVISÃO
%  -> MÓDULO (RESTO DA DIVISÃO)
```

Operadores relacionais

```
>    -> MAIOR
>=   -> MAIOR OU IGUAL
<    -> MENOR
<=   -> MENOR OU IGUAL
==   -> IGUAL
!=   -> DIFERENTE
```

Operadores lógicos

```
&&   -> E
||   -> OU
!    -> NEGAÇÃO
```

Estrutura de condição

```
var media : Float = 8.5

if media >= 7 {
    print("Aprovado(a)")
} else if media >= 5 {
    print("Recuperação")
} else {
    print("Reprovado(a)")
}
```

Operador ternário

```
var idade : Int = 22

var situacaoIdade = (idade >= 18)
? "Maior de idade" : "Menor de idade"

print(situacaoIdade)
```

Estrutura de escolha

```
var produto : String = "iPhone 17"

switch produto {
    case "Samsung Galaxy S26":
        print("O produto Samsung Galaxy S26 - R$8.500,00")

    case "MacBook Pro":
        print("O produto MacBook Pro - R$16.700,00")

    case "iPhone 17":
        print("O produto iPhone 17 - R$11.200,00")

    default:
        print("Produto desconhecido")
}
```

Laço while (enquanto)

```
var indice : Int = 0

while indice <= 10 {
    print(indice)

    indice += 1
}
```

Laço repeat (faça / enquanto)

```
var indice : Int = 0

repeat {
    print(indice)
    indice += 1
} while indice <= 10
```

Laço for (para)

```
for indice in 0...10 {
    print(indice)
}
```

Vetor

```
var nomes1 = ["Ana", "Bianca", "Carlos"]
var nomes2 : [String] = ["Ana", "Bianca", "Carlos"]
```

Laço forEach (para cada)

```
nomes.forEach { print($0) }
```

Funções

```
func apresentacao(nome: String) -> Void{
    print("Bom dia \(nome)")
}
```

Funções anônimas (Closures)

```
let soma: (Int, Int) -> Int = { $0 + $1 }
print(soma(10, 7))
```

Orientação a Objetos

Classe, atributo, método e objeto

```
class Pessoa {

    var nome: String = ""
    var idade: Int = 0

    func apresentacao() -> Void {
        print("Bom dia \(nome), você tem \(idade) anos.")
    }

}

let p = Pessoa()
p.nome = "Juliana"
p.idade = 32
p.apresentacao()
```

Encapsulamento

```
class Pessoa {
    private var _nome: String = ""
    private var _idade: Int = 0

    // 1ª maneira
    var nome: String {
        get { return _nome }
        set (valor) { _nome = valor }
    }

    // 2ª maneira
    var idade: Int {
        get { return _idade }
        set { _idade = newValue }
    }

    // 3ª maneira (READONLY FORA DA CLASSE - GET)
    private(set) var cidade: String = "São Paulo"
}
```

Modificadores de acesso

1. PUBLIC (Atributos e métodos disponíveis para acesso de qualquer local)

2. INTERNAL (Acesso a atributos e métodos no mesmo módulo/pacote)

3. FILEPRIVATE (Atributos e métodos disponíveis apenas no mesmo arquivo)

4. PRIVATE (Atributos e métodos disponíveis apenas na mesma classe)

Construtor e destrutor

```
class MinhaClasse {
    // Construtor
    init() {
        print("Construtor em ação!")
    }

    // Destrutor
    deinit {
      print("Destrutor em ação!")
    }
}
```

Atributos e métodos estáticos

```
class Calculos {

    static var pi: Float = 3.14

    static func soma(_ numero1: Int, _ numero2: Int) -> Int {
        return numero1 + numero2
    }

}
```

Herança

```
// Classe pai
class Pessoa {
    var nome: String = ""
}

// Classe filho
class Colaborador:Pessoa {
    var salario: Double = 0.0
}
```

Polimorfismo - Sobrecarga

```
class Soma {

    func somar(numero1: Int, numero2: Int) {
        print(numero1 + numero2)
    }

    func somar(numero1: Int, numero2: Int, numero3: Int) {
        print(numero1 + numero2 + numero3)
    }

}
```

Polimorfismo - Sobrescrita

```
class Descontos {

    func impostoRenda(salario: Double) {
        print("Desconto padrão do Imposto de Renda: R$ \(salario * 0.10)")
    }

}

class DescontosDesenvolvedor: Descontos {

    override func impostoRenda(salario: Double) {
        print("Desconto do Imposto de Renda para desenvolvedores: R$
            \(salario * 0.12)")
    }

}
```

Extras

ENUMS

```
enum Linguagens {
    case Swift
    case Java
    case Python
}
```

Pattern Matching

```
enum Status {
    case planejamento(descricao: String)
    case desenvolvimento(tecologia: String)
    case entrega(versao: Float)
}
```

Guard Let

```
func mensagem(_ nome: String?){

    guard let nomeRetorno = nome else {
        print("O nome não pode ser nulo!")
        return
    }

    print("Olá \(nomeRetorno)")

}
```

Tupla

```
var pessoa1 = ("Alice", 28)
print(pessoa1.0)

var pessoa2 = (nome: "Alice", idade: 28)
print(pessoa2.nome)
```

Converter dados

```
var numero1: String = "42"

var numero2: Int = Int(numero1) ?? 0
print(3 + numero2)
```

Struct

```
struct Pessoa {
    var nome: String = ""
}
```

Protocol

```swift
// Protocol de atributo
protocol Entidade {
    var nome: String { get }
}

// Protocol de função
protocol Funcao {
    func mensagem() -> Void
}

// Struct que implementa os protocols
struct MeuStruct: Entidade, Funcao {
    var nome: String

    func mensagem() {
        print("Hello Swift!")
    }
}
```

Extension

```
// Struct
struct Mensagem {
    func mensagem1() -> Void {
        print("Mensagem 1")
    }
}

// Extension
extension Mensagem {
    func mensagem2() -> Void {
        print("Mensagem 2")
    }
}
```

Tratativa de exceção

```
//1º Definir o erro
enum ErroDeDivisao: Error {
    case divisaoPorZero
}
```

Tratativa de exceção

```
//2º Função que terá acesso ao Enum
func dividir(_ numero1: Int, _ numero2: Int) throws -> Int {
    if(numero2 == 0) {
        throw ErroDeDivisao.divisaoPorZero
    }

    return numero1 / numero2
}
```

Tratativa de exceção

```
//3 Executar função
do {
    let resultado = try dividir(10, 0)
    print(resultado)
} catch {
    print("Falha \(error)")
}
```

Generics

```
func verificarTipo<T>(_ valor: T) -> Void {

    // Condicional
    if(valor is String) {
        print("Valor do tipo String")
    } else if(valor is Int) {
        print("Valor do tipo Int")
    } else {
        print("O valor é diferente de String ou Int")
    }

}
```

Funções assíncronas

```
import Foundation

func tarefa1() async {
    try? await Task.sleep(for:.seconds(2))
    print("Tarefa 1")
}

func tarefa2() async {
    //Thread.sleep(forTimeInterval: 5)
    try? await Task.sleep(for:.seconds(5))
    print("Tarefa 2")
}
```

Funções assíncronas

```
Task {
    // Executar cada tarefa e armazenar em uma referência
    async let t1: () = tarefa1()
    async let t2: () = tarefa2()

    // Exibir os prints
    await t1
    await t2
}
```

Materiais complementares

Materiais complementares

- Apple Developer
- Docs Swift
- GeeksforGeeks
- Tutorials Point